

Music Genre Classification: Bridging Signals & Machine Learning

Overview:

The Problem: Raw audio is highly complex and cannot be fed directly into a standard artificial intelligence model.

The Objective: To build an automated system that listens to a 30-second audio clip, mathematically analyzes its physical soundwave properties, and accurately predicts its musical genre.

The Approach: We combine Signals & Systems (SnS) mathematics to translate audio into structured data, followed by Machine Learning (ML) algorithms to recognize patterns in that data.

Project Roadmap:

To effectively manage this complexity, we have divided the entire project into 4 distinct phases, progressing from raw audio files to a fully trained AI model.

Project Roadmap & Phases

Phase 1: Data Ingestion & Feature Extraction

Loading raw .wav files and dividing them into localized time-segments.

Applying mathematical transformations (STFT, MFCCs) to extract frequency, time, and human-hearing features. (This is the Python code we have completed).

Phase 2: Exploratory Data Analysis & Preprocessing

Visualizing the extracted data to find patterns (e.g., comparing the noisiness of Metal vs. Classical).

Scaling the numbers and cleaning the dataset so it is perfectly formatted for AI training.

Phase 3: Machine Learning Model Development

Feeding our extracted mathematical features into Machine Learning algorithms.

Training baseline models (like Random Forest or SVM) to classify the 10 distinct genres.

Phase 4: Model Evaluation & Optimization

Testing the AI on new, unseen songs to see how accurately it predicts the genre.

Tuning the model's settings to improve performance and generating final visual reports (like Confusion Matrices).

Code Snippet — Part 1

```
import os
import librosa
import librosa.display
import numpy as np
import pandas as pd
import warnings
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings('ignore')

# 1. Define the SnS Parameters
DATASET_PATH = "genres_original"
CSV_PATH = "music_features.csv"
SPEC_FOLDER = "spectrograms"
SPECS_PER_GENRE = 2
SAMPLE_RATE = 22050
TRACK_DURATION = 30
SAMPLES_PER_TRACK = SAMPLE_RATE * TRACK_DURATION
NUM_SEGMENTS = 10
SAMPLES_PER_SEGMENT = int(SAMPLES_PER_TRACK / NUM_SEGMENTS)
SKIP_FILES = {"jazz.00054.wav"}

def extract_features(dataset_path):
    data = {
        "song_id": [], "length": [],
        "chroma_stft_mean": [], "chroma_stft_var": [],
        "rms_mean": [], "rms_var": [],
        "spectral_centroid_mean": [], "spectral_centroid_var": [],
        "spectral_bandwidth_mean": [], "spectral_bandwidth_var": [],
        "rolloff_mean": [], "rolloff_var": [],
        "zero_crossing_rate_mean": [], "zero_crossing_rate_var": [],
        "harmony_mean": [], "harmony_var": [],
        "perceptr_mean": [], "perceptr_var": [], "tempo": []
    }
```

Part 1 – Setting up the parameters and data structure

The code imports the necessary programming tools for audio processing, math, and creating images.

It defines the basic settings for the project: the sampling rate (22,050 numbers per second of audio), the length of the tracks (30 seconds), and how many segments to divide each track into (10 segments of 3 seconds each).

It sets the names of the folders where the dataset lives and where the final results will be saved, as well as a list of known broken files to skip.

2. Creating the data structure

It creates a large, empty dictionary to act as the blueprint for the final data table.

It lists out the exact names of the features it will calculate (like length, tempo, and spectral centroid).

It uses a loop to automatically generate 40 empty columns for the MFCC features (20 for the averages, and 20 for the variances) so they don't have to be typed manually.

3. Preparing the output folders

The code creates a new folder on the computer specifically to store the spectrogram images it will generate later.

It sets up tracking variables to count how many images have been saved for each genre, ensuring it only saves a specific amount to save storage space.

Code Snippet — Part 2

```
for i, (dirpath, dirnames, filenames) in enumerate(os.walk(dataset_path)):
    dirnames.sort()
    if dirpath != dataset_path:
        genre_label = os.path.basename(dirpath)
        print(f"\nProcessing SnS computations for genre: {genre_label}")

        for f in sorted(filenames):
            if not f.endswith('.wav') or f in SKIP_FILES:
                continue
            file_path = os.path.join(dirpath, f)

            try:
                signal, sr = librosa.load(file_path, sr=SAMPLE_RATE)
            except Exception as e:
                print(f"Error loading {file_path}. Skipping.")
                continue

            for d in range(NUM_SEGMENTS):
                start = SAMPLES_PER_SEGMENT * d
                finish = start + SAMPLES_PER_SEGMENT
                y = signal[start:finish]

                if len(y) < SAMPLES_PER_SEGMENT:
                    continue
```

Part 2: Sorting, filtering, and segmenting the audio

1. Going through the folders and getting the genre names

The code looks inside the main dataset folder and finds all the sub-folders inside it.

It sorts these sub-folders in alphabetical order so that it processes the data in the exact same order every time you run the script.

When it enters a sub-folder, it reads the name of that folder (for example, "blues" or "pop") and saves it as the genre label. It will attach this label to all the songs it finds inside.

2. Sorting and filtering the files

Inside the genre folder, it takes all the files and sorts them alphabetically.

Before it tries to process a file, it checks two things:

1. Does the file name end in .wav? (This ensures it is actually an audio file).
2. Is the file name on our SKIP_FILES list? (This ensures we don't process known broken files).

If a file doesn't meet these rules, the code ignores it and moves on to the next file.

3. Loading the audio into memory

The code then reads the audio file and converts the sound into a long list of numbers based on our sample rate (22,050 numbers for every second of audio).

This step is wrapped in a "try and except" block. This means if a file is corrupted and fails to load, the program will just print an error message and move on to the next track, rather than crashing the entire script.

4. Dividing the song into smaller segments

Because analyzing a full 30-second song all at once is too broad, the code uses a loop to divide the song into 10 smaller, equal segments (3 seconds each).

It calculates the exact starting and stopping points for each segment and extracts that specific section of the audio to analyze separately.

Finally, it checks the length of each segment. If a piece of the audio is shorter than the required 3 seconds (which usually happens at the very end of a track), the code skips that piece. This ensures that every piece of data passed to the machine learning model later is exactly the same size.

Code Snippet — Part 3

```
stft = np.abs(librosa.stft(y))

    if d == 0 and saved_count.get(genre_label, 0) < SPECS_PER_GENRE:
        mel = librosa.feature.melspectrogram(y=y, sr=sr)
        mel_db = librosa.power_to_db(mel, ref=np.max)
        fig, ax = plt.subplots(figsize=(6, 3))
        img = librosa.display.specshow(mel_db, sr=sr, x_axis="time",
                                       y_axis="mel", ax=ax)
        fig.colorbar(img, ax=ax, format="%+2.0f dB")
        ax.set_title(f"{genre_label} - {f}")
        fig.tight_layout()
        fig.savefig(os.path.join(SPEC_FOLDER, f"{f[:4]}.png"), dpi=120)
        plt.close(fig)
        if genre_label not in grid_specs:
            grid_specs[genre_label] = mel_db
        saved_count[genre_label] = saved_count.get(genre_label, 0) + 1

data["song_id"].append(f)
data["length"].append(len(y))

chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr)
data["chroma_stft_mean"].append(np.mean(chroma_stft))
data["chroma_stft_var"].append(np.var(chroma_stft))

rms = librosa.feature.rms(y=y)
data["rms_mean"].append(np.mean(rms))
data["rms_var"].append(np.var(rms))

spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr)
data["spectral_centroid_mean"].append(np.mean(spec_cent))
data["spectral_centroid_var"].append(np.var(spec_cent))

spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)
data["spectral_bandwidth_mean"].append(np.mean(spec_bw))
data["spectral_bandwidth_var"].append(np.var(spec_bw))
```

Part 3: Visualizing and calculating frequency features

1. Generating and saving visual charts (Spectrograms)🌀

For the first segment of the first few songs in each genre, the code calculates a visual representation of the audio frequencies (a Mel-spectrogram).

It formats this chart with a color scale, labels, and a title containing the genre and file name.

It saves this chart as an image file in the spectrogram folder and immediately closes it so it doesn't use up the computer's memory.

2. Logging basic segment information

The code records the exact file name and the length of the audio segment into the empty dictionary created in the first step.

3. Calculating frequency-based features

The code applies mathematical formulas to the 3-second audio segment to extract specific characteristics about the frequencies present in the sound.

It calculates values for pitch intensity (chroma), overall loudness (RMS), the center of mass of the frequencies (spectral centroid), the spread of the frequencies (bandwidth), and the point where most of the high frequencies drop off (rolloff).

4. Storing the averages and variances

These formulas generate a large sequence of numbers for the 3-second segment. The code calculates the average (mean) and how much those numbers vary (variance) for each feature.

It saves these two final numbers into the data dictionary.

Code Snippet — Part 4

```
zcr = librosa.feature.zero_crossing_rate(y)
data["zero_crossing_rate_mean"].append(np.mean(zcr))
data["zero_crossing_rate_var"].append(np.var(zcr))

harmony, perceptr = librosa.effects.hpss(y)
data["harmony_mean"].append(np.mean(harmony))
data["harmony_var"].append(np.var(harmony))
data["perceptr_mean"].append(np.mean(perceptr))
data["perceptr_var"].append(np.var(perceptr))

tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
data["tempo"].append(float(np.atleast_1d(tempo)[0]))

mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
for b in range(20):
    data[f"mfcc{b+1}_mean"].append(np.mean(mfcc[b]))
    data[f"mfcc{b+1}_var"].append(np.var(mfcc[b]))

data["label"].append(genre_label)

if grid_specs:
    names = sorted(grid_specs)
    fig, axes = plt.subplots(2, 5, figsize=(20, 7))
    for ax, name in zip(axes.ravel(), names):
        librosa.display.specshow(grid_specs[name], sr=SAMPLE_RATE, x_axis="time",
                                y_axis="mel", ax=ax)

        ax.set_title(name)
    for ax in axes.ravel()[len(names):]:
        ax.axis("off")
    fig.tight_layout()
    fig.savefig(os.path.join(SPEC_FOLDER, "all_genres_grid.png"), dpi=120)
    plt.close(fig)

return pd.DataFrame(data)

if __name__ == "__main__":
    df = extract_features(DATASET_PATH)
```

Part 4: Calculating time/hearing features and exporting

1. Calculating time and rhythm features

The code calculates the zero-crossing rate, which measures how often the audio signal shifts from positive to negative (indicating how noisy the sound is).

It mathematically separates the melodic sounds from the percussive drum sounds (HPSS) and measures both.

It calculates the estimated beats-per-minute (tempo) of the audio segment and ensures it is saved as a standard decimal number.

3. Attaching the final label

It adds the correct genre label to this specific row of data so the machine learning model knows the correct answer for this segment.

2. Calculating human-hearing features (MFCCs)

The code calculates 20 Mel-Frequency Cepstral Coefficients (MFCCs), which are standard mathematical values used to represent how the human ear perceives the shape of the sound.

Just like in the previous steps, it records the average and variance for all 20 of these numbers and adds them to the dictionary.

4. Creating a summary image and exporting the data

Once all files in the entire dataset are processed, the code takes one saved chart from each genre and combines them into a single grid image, saving it for the final report.

Finally, it converts the massive dictionary of calculated numbers into a structured table format and saves it as a .csv file on the computer.